

Programação Funcional em Haskell

Estruturas persistentes

REALIZAR APÓS posição 62: 16.7 Estruturas de Dados Puramente Funcionais - Árvores Rubro-Negras: Inserção

Posições na playlist 54 a 62 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |
<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Persistência; listas; árvores; DFS/BFS; zipper; rose trees; rubro-negras

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Persistência

(a) Compare atualizar lista/árvore funcional criando nova versão com alterar objeto mutável. (b) Desenhe compartilhamento ao inserir cabeça numa lista. (c) Discuta custo de manter várias versões e por que compartilhar estruturas imutáveis é seguro.

Desenvolvimento / justificativa / esquema:

Árvore recursiva

Considere data $Arvore\ a = Vazia | No\ a(Arvore\ a)(Arvore\ a)$. (a) Implemente percurso em-ordem e conte nós. (b) Descreva BFS com fila. (c) Compare ordem e custo, considerando que concatenação ingênua pode elevar custo.

Desenvolvimento / justificativa / esquema:

Zippers e rose trees

(a) Explique contexto necessário para subir em zipper de árvore binária. (b) Dê informação guardada ao descer à esquerda. (c) Generalize conceitualmente para rose tree com lista de filhos e descreva reconstrução após editar um foco.

Desenvolvimento / justificativa / esquema:

Rubro-negra persistente

(a) Enuncie invariantes rubro-negras e explique por que controlam altura. (b) Faça inserção 10, 20, 30, dando árvore balanceada. (c) Explique quais caminhos/nós precisam ser reconstruídos numa implementação puramente funcional e como validar versões antigas.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Persistência (15 pontos)

Nova lista $x:xs$ cria um nó e compartilha toda xs ; versão anterior continua válida. Árvore persistente copia caminho alterado e compartilha subárvores restantes. Imutabilidade evita que atualização destrua visão de versão antiga. Custo adicional depende dos caminhos copiados e versões mantidas; persistência não significa copiar tudo nem usar memória zero.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Árvore recursiva (25 pontos)

em $Vazia=[]$; em $(No\ x\ l\ r)=em\ l++[x]++em\ r$. conta $Vazia=0$; conta $(No\ _\ l\ r)=1+conta\ l+conta\ r$. BFS usa fila, retira nó e acrescenta filhos não vazios. Visita de nós é $O(n)$, mas concatenação em listas pode tornar percurso ingênuo $O(n^2)$ numa árvore degenerada; usar acumulador/lista de diferenças evita custo repetido. BFS precisa de fila eficiente, não `append` indiscriminado à frente.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Zippers e rose trees (25 pontos)

Contexto contém valor do pai, subárvore irmã e lado de onde veio, mais trilha dos ancestrais. Ao descer à esquerda, guardar valor do pai e subárvore direita. Subir reconstrói nó com foco atualizado e irmã preservada. Rose tree precisa guardar irmãos anteriores/posteriores e contexto do pai para reconstruir lista de filhos. A atualização cria novos nós no caminho, compartilhando partes não modificadas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Rubro-negra persistente (35 pontos)

Raiz/NIL pretos; nenhum vermelho com filho vermelho; altura negra igual. Caminho tem no máximo dois nós por nível negro, limitando altura logarítmica. Após correção, raiz 20 preta, filhos 10 e 30 vermelhos. Em persistente, reconstruir caminho de busca e região de rotação/recoloração, compartilhando demais subárvores. Testar ordem em ordem, invariantes e que versões anteriores preservam suas sequências após nova inserção.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.